University Management System

Agile Software Engineering Project — Phase 2

MVP Implementation, Sprint Execution & Meeting Minutes

Team Members

Yehia Mohamed — 23P0067

Omar Tamer — 23P0096

Saeed Mohamed — 23P0255

Course: CSE342 — Agile Software Engineering

Instructor: Dr. Mohamed H. ElGazzar

Date: May 2026

Table of Contents

1. Introduction

This document is the Phase 2 submission for our University Management System (UMS) project. In Phase 1 we focused on planning — writing user stories, prioritizing the backlog with MoSCoW, estimating with Planning Poker, and setting up Jira. In Phase 2 we actually built the thing.

Sprint 1 ran for two weeks and we committed to all 10 Must Have stories (46 story points). The goal of this document is to walk through what we did, the meetings we held, how the MVP came together, and where things stand now.

Most of the planning artefacts (MoSCoW, INVEST, story splitting, DoD, estimation) were defined in Phase 1, so we summarise them here and refer back to the Phase 1 document for the full reasoning.

2. Prioritization Technique — MoSCoW

We picked MoSCoW back in Phase 1 and stuck with it for Phase 2. The reason is honestly pretty simple — it's easy to apply, it tells you exactly what your MVP is (everything in Must Have), and it fits naturally with how Scrum sprints work. You pull the Musts in first, then the Shoulds if you have room.

2.1 How We Used It in Phase 2

During the sprint we kept the MoSCoW labels visible on every Jira card. Whenever we had to make a trade-off — for example when US-08 (material upload) took longer than estimated — the labels reminded us that finishing the Musts mattered more than starting on the Shoulds. We also re-checked priorities at the start of the sprint to make sure nothing had shifted since Phase 1. Nothing had.

2.2 Sprint 1 Distribution

- Must Have: 10 stories — all pulled into Sprint 1 (the MVP).
- Should Have: 6 stories — kept in the backlog for Sprint 2.
- Could Have: 2 stories (US-13, US-14) — backlog, lower priority.
- Won't Have: nothing for now. Some features are clearly later-sprint work but we didn't permanently exclude anything.

3. MVP Features (Product Backlog Items in the MVP)

Our MVP is the smallest version of the UMS that the three main user types (admin, professor, student) can actually use. It covers all four modules at a basic level — Facilities, Curriculum, Staff, and Community — plus the cross-cutting login system. The 10 Must Have stories below make up the MVP.

ID	User Story (short form)	Module	Priority	Pts
US-01	Admin manages classroom & lab records (CRUD)	Facilities	Must	5
US-02	Professor views availability & books a room	Facilities	Must	5
US-04	Admin manages student records (CRUD)	Facilities	Must	5
US-06	Admin defines course catalog (core/elective)	Curriculum	Must	5
US-07	Student browses courses & registers for electives	Curriculum	Must	5

ID	User Story (short form)	Module	Priority	Pts
US-08	Professor uploads assignments & materials	Curriculum	Must	5
US-10	Student views grades & feedback	Curriculum	Must	3
US-11	Admin manages staff directory (Profs/TAs)	Staff	Must	5
US-17	Admin posts university-wide announcements	Community	Must	3
US-18	User logs in with role-based access	Cross-cutting	Must	5

Total: 46 story points across 10 stories. Each story has acceptance criteria documented in the Phase 1 backlog, and those criteria were used as the test checklist when reviewing each one.

3.1 What the MVP Actually Lets You Do

- Log in as admin, professor, or student and see only the menu items for your role.
- As admin: manage rooms, students, courses, staff, and post announcements.
- As professor: book rooms by date/time, upload materials to your courses, see staff/announcements.
- As student: browse and register for electives, view grades and feedback, see staff/announcements.
- Data persists in the browser (localStorage) so refreshing doesn't wipe your work.

4. How Epics & Features Were Split into Stories

The original project description gave us four big modules (Facilities, Curriculum, Staff, Community). Each of those is basically an epic and each one needed to be broken down into stories that are small enough to estimate and finish in one sprint. We used three of the techniques from the lectures: Workflow Steps, Data Operations (CRUD), and Business Rules. This is the same approach we used in Phase 1 — repeating the examples here for completeness.

4.1 Why Splitting Matters

If a story is too big it becomes hard to estimate, hard to finish, and you can't see your progress. We aimed for stories small enough that one person can mostly own them within a sprint, but not so tiny that the backlog becomes a mess of trivial tickets.

4.2 Example 1 — Administrative Office Automation (Workflow Steps)

The brief described “managing student records, generating transcripts, and handling admission applications” as one feature. These are three different workflows, so we split by workflow steps:

1. US-04: Manage student records (CRUD) — went into Sprint 1.
2. Generate transcripts — backlog, future sprint.
3. Handle admission applications — backlog, future sprint.

Each one can be built and tested independently, which is exactly what we want.

4.3 Example 2 — Classroom & Lab Management (CRUD + Business Rules)

The original requirement bundled room management, booking, availability, and maintenance reporting. We split using Data Operations and Business Rules:

4. US-01: Room records CRUD — basic data ops.
5. US-02: View availability & book — needs scheduling rules and conflict checking.
6. US-03: Report maintenance — different data and workflow, became Should Have.

This let US-01 and US-02 ship in Sprint 1 as Musts, while US-03 stays in the backlog without blocking anything.

4.4 Example 3 — Login & Role-Based Access (Cross-cutting + Sub-tasks)

US-18 is technically one user story ("log in with role-based access") but it touches every part of the system. So instead of splitting it into more stories, we kept it as one story and used Jira sub-tasks for the technical pieces:

- Design login page UI.
- Set up authentication backend (in our case, validate against the user list in localStorage).
- Implement role-based routing (different sidebar per role).
- Write basic tests.

5. Estimation Techniques — Planning Poker

We used Planning Poker for all our estimates, both in Phase 1 and again whenever we re-estimated during the sprint. The Fibonacci scale (1, 2, 3, 5, 8, 13) is the one we used because the gaps between numbers force you to commit — you can't sit on the fence between a 5 and a 6, you pick 5 or 8.

5.1 How a Planning Poker Round Worked for Us

7. Yehia (PO) reads the story and acceptance criteria out loud.
8. Anyone can ask clarifying questions.
9. All three of us pick a card privately (we used a free online poker tool).
10. We reveal at the same time.
11. If the votes match — done, that's the estimate.
12. If they don't match, the highest and lowest voters explain their reasoning, then we re-vote.

Most stories converged after one or two rounds. The biggest disagreement was on US-02 (room booking) — Omar voted 8 because of the double-booking logic, Saeed voted 5 because the data structure was simple. After Omar explained the conflict-checking edge cases, we agreed on 5 because we realised we could prevent double-booking with a single check rather than a full scheduling engine.

5.2 Why Story Points Instead of Hours

Hours feel exact but they're not. Two devs of different speeds will quote different hours for the same work, and nobody's actually that good at predicting how long things take. Story points measure relative size — "is this story bigger or smaller than that one?" — which the team can agree on more reliably. Velocity (story points done per sprint) then tells us our actual capacity over time.

6. Meeting Minutes — Scrum Events for Sprint 1

This section captures all the Scrum events we held during Sprint 1. We tried to keep these minutes short but specific — names, story IDs, real things people said. Where the topic was already covered in Phase 1 (like the 7-step sprint planning) we summarise rather than repeat.

6.1 Sprint Planning Meeting

Date: 21 April 2026 | Time-box: 4 hours | Attendees: Yehia, Omar, Saeed

Sprint planning followed the 7 steps from Phase 1 (set the stage → define sprint goal → present backlog → break down stories → estimate → check capacity → finalise commitment). We won't repeat all the details here since they're in Phase 1 Section 7, but here are the agreement points that came out of the meeting:

Agreement Points

- Sprint Goal: Build the foundation of the UMS — login with role-based access, room management & booking, student records, course catalog & registration, material upload, grade viewing, staff directory, announcements.
- Sprint duration: 2 weeks (21 April → 4 May 2026).
- Sprint backlog: 10 Must Have stories, 46 story points total.
- Capacity: ~10–12 hours/week per person × 3 people × 2 weeks ≈ 60–72 hours of dev time.
- Story assignments — Yehia: US-01, US-02, US-04 (Facilities). Omar: US-06, US-07, US-08 (Curriculum). Saeed: US-10, US-11, US-17 (Staff & Community). US-18 (login) shared by all three because it's cross-cutting.
- Tech stack agreed: HTML + CSS + vanilla JS, localStorage for persistence, no backend. We had a short debate about using Node + Express but decided against it — too much overhead for a 2-week sprint and the brief doesn't require a backend.
- Daily standups will be at 7pm (after classes) on a WhatsApp call, max 15 minutes.

Open Questions / Risks Raised

- Saeed asked whether materials in US-08 should be actually stored or just referenced. Decision: store as base64 in localStorage for now (small files only). Real file storage is a Sprint 2 concern.
- Omar flagged that grade data (US-10) needs to come from somewhere even though we don't have grade-entry yet. Decision: seed demo grade data on first load so the student account has something to view.

6.2 Daily Standups

We held daily standups every weekday over WhatsApp call at 7pm. Each person answered the three standard questions: what did I do yesterday, what will I do today, what is blocking me. Below are four representative standups from the sprint — we picked these because they capture the actual rhythm of the sprint (early enthusiasm, mid-sprint drag, near-the-end push).

Daily Standup #1 — Tuesday 22 April 2026

Attendees: Yehia, Omar, Saeed (all present, ~10 min)

Yehia: Yesterday set up the project folder and the login page HTML structure with Saeed (we paired on the shell). Today I'll start US-01 — the room CRUD. No blockers.

Omar: Yesterday I sketched out the data model for courses on paper. Today starting US-06 (course catalog admin). One question — do we want the same modal pattern Yehia is using for rooms, for consistency? Yehia confirmed yes.

Saeed: Yesterday I built the login form and the seed-data initialiser (the three demo accounts). Today I'll wire up the role-based sidebar so each role sees different menu items. Blocker: I'm not sure yet how to lay out the sidebar — will look at a few examples and pick one.

Daily Standup #2 — Friday 25 April 2026

Attendees: Yehia, Omar, Saeed

Yehia: US-01 is done. Started US-02 (room booking) yesterday. The double-booking check is trickier than I expected — I'm comparing date+time slots and need to handle overlaps not just exact matches. Will finish today.

Omar: US-06 done. US-07 (student browse & register) is about 60% done — the filter by type works, registration works, but I still need to add the prevent-double-registration check. Today I'll close that out.

Saeed: Sidebar and dashboard done — US-18 acceptance criteria are met. Started US-11 (staff directory) yesterday. Going well. Today I'll finish the admin CRUD side and move on to the search/view side that all roles share.

(Omar as Scrum Master): Reminder we're 4 days in and 3 stories are fully done. We're roughly on track — keep going.

Daily Standup #3 — Tuesday 29 April 2026

Attendees: Yehia, Omar, Saeed

Yehia: US-02 done finally — the booking conflict logic took longer than the points suggested, but it works. Started US-04 (student records). Should be quick since it's similar to the room CRUD pattern.

Omar: US-07 done. Yesterday started US-08 (material upload). Storing files in localStorage as base64 like we agreed. The file picker works, just need to wire up the “view per course” side.

Saeed: US-11 done. Started US-17 (announcements). Pretty straightforward, the reverse chronological sort is one line. Should finish today and start US-10 (grades) tomorrow.

Issue raised: Yehia mentioned the styling is starting to drift between modules — three of us are writing slightly different CSS. Decision: Saeed (who owns the main stylesheet) will do a 30-min consistency pass on Thursday.

Daily Standup #4 — Friday 2 May 2026

Attendees: Yehia, Omar, Saeed

Yehia: US-04 done. All my Sprint 1 stories are now in In-Review. Today I'll review Omar and Saeed's work and write up the testing notes.

Omar: US-08 done as of last night. All 3 of my Curriculum stories are done. Going to spend today helping wherever needed and double-checking the demo flow before the review on Monday.

Saeed: US-17 and US-10 both done. Did the CSS consistency pass yesterday. Honestly feeling good — the whole MVP runs end-to-end now. Today: write the README, push everything to GitHub, prepare the demo script for the sprint review.

Mood check: team agreed we're going to make the sprint goal. Sprint review scheduled for Monday.

6.3 Sprint Review

Date: 5 May 2026 | Time-box: 2 hours | Attendees: Yehia, Omar, Saeed (and our TA acted as stakeholder)

The Sprint Review is where we showed off what we actually built. We ran the demo on Saeed's laptop with the live website, going through each story one by one. Everyone took turns demoing the stories they owned.

Demo Walkthrough

- Saeed demoed US-18 (login) by signing in as all three demo accounts and showing the sidebar change for each role.
- Yehia demoed US-01 by adding a new lecture hall, editing it, then deleting it (confirmation dialog appeared as required by AC4).
- Yehia demoed US-02 by booking a room for a specific date/time, then trying to book the same slot again — system blocked it, as required by AC3.
- Yehia demoed US-04 by adding a student, searching for them, updating, and deleting.
- Omar demoed US-06 by adding both a core and an elective course and showing they appear in separate sections.
- Omar demoed US-07 by browsing as a student, filtering by type, registering for an elective, and trying to register again (blocked).
- Omar demoed US-08 by uploading a PDF as a professor and viewing it as the student account.
- Saeed demoed US-10 by showing the seeded grade data per course with feedback comments.
- Saeed demoed US-11 by adding a TA as admin, then logging in as student to search and view the staff directory.
- Saeed demoed US-17 by posting two announcements with different dates and confirming the latest appears at the top.

Stakeholder Feedback (TA)

- Positive: all 10 acceptance criteria sets are met, the role-based access works smoothly, and the UI is consistent.
- Suggestion: in Sprint 2 the announcements page could allow editing/deleting posts (currently it's create-only).
- Suggestion: error messages on login could be a bit more visible — currently the red text is small.
- Question raised: how are we storing files for materials? We explained the base64-in-localStorage approach and that real storage will come with a backend later. TA accepted this for the MVP scope.

Sprint Outcome

- All 10 stories accepted as Done.
- Velocity for Sprint 1: 46 story points (matches commitment).
- Two new items added to backlog from feedback: announcement edit/delete, login error message visibility improvement.

6.4 Sprint Retrospective

Date: 5 May 2026 (right after the review) | Time-box: 1 hour | Attendees: Yehia, Omar, Saeed

We used the simple “What went well / What didn't / What to improve” format. Everyone wrote on a shared doc for 5 minutes, then we discussed.

What Went Well

- We hit every Must Have. 46/46 story points done.
- Daily standups at 7pm worked — short, consistent, and we caught problems early (like the styling drift).
- Pair-programming on the login/shell at the start of the sprint set a good consistent base for the rest.
- Planning Poker estimates were mostly accurate. The biggest miss was US-02 which felt more like an 8 than a 5 in practice — noted for Sprint 2.

What Didn't Go Well

- Three people writing CSS in parallel led to inconsistencies. Saeed had to spend ~30 mins fixing things mid-sprint.
- US-02 (room booking) overran the estimate — the conflict-checking logic is more involved than we thought.
- Sub-task breakdown was only done for US-18. The other stories went straight from “To Do” to “In Progress” without intermediate tasks, which made it harder to see progress on the board.
- Initial commit was huge because we didn't push frequently in the first 3 days. We got better at this later but it made the early Git history less useful.

What to Improve in Sprint 2

13. Style guide first. Saeed will write a one-page CSS conventions doc (variables, spacing scale) before anyone writes a new module.
14. Add sub-tasks to every story bigger than 3 points. Makes the board more useful and forces us to think through the implementation upfront.
15. Re-estimate the “booking-style” stories (anything with conflict checking) more conservatively — start from 8 not 5.
16. Push commits at least once per work session — not the giant dumps we had at the start.
17. Rotate roles for Sprint 2: Omar → Product Owner, Saeed → Scrum Master, Yehia → Developer (per the course instruction to rotate).

7. Definition of Done

Our DoD is what the team agreed “finished” actually means for any story. This is the same DoD we set in Phase 1 — we re-confirmed it at sprint planning and didn't change it during the sprint. A story is Done when:

18. The feature meets all the acceptance criteria for that story.
19. The code is reviewed, approved, and merged into the main branch in our Git repository.
20. Basic testing has been done and the feature has no obvious bugs.
21. Both functional and non-functional requirements are met (the feature works correctly and the UI is usable).
22. Any necessary documentation is updated (e.g. README, comments on top of each module).
23. The Product Owner has reviewed and accepted the feature.
24. The Jira ticket is moved to Done and the feature has been demonstrated during the sprint review.

We deliberately kept the DoD realistic for a 3-person student team. We didn't include CI/CD pipelines or staging deployment because that's not where we are. The retrospective will be the moment to tighten the DoD when we're ready.

8. Project Links

8.1 Jira Board

All 18 backlog items are on the Jira board, with Sprint 1 (10 Must Have stories, 46 SP) marked Done.

Jira: https://yehyaelabyad.atlassian.net/jira/software/projects/UMS/boards/34

8.2 GitHub Repository

All Sprint 1 code is in the repository below. The README.md explains how to run the project (just open index.html in a browser — no build step). Each user story is referenced in code comments.

GitHub: https://github.com/yehyaelabyad/university-management-system

Note: replace the link above with the final URL once the repo is pushed (it should already match this if the team followed the repo-name convention).

9. Anticipating Changes (Sprint 2 Outlook)

Agile is supposed to absorb change, not resist it. Coming out of Sprint 1, here's where we expect things to evolve in Sprint 2:

- Stakeholder feedback added two new items (announcement edit/delete, better login error visibility). Both will be reprioritised at Sprint 2 planning.
- US-02 turned out heavier than estimated. We'll re-baseline anything similar (booking, scheduling, conflict logic) at the start of next sprint.
- Should-Have stories (US-03, US-05, US-09, US-12, US-15, US-16) are the candidates to pull in next. We expect to be able to complete around 35–45 points based on Sprint 1 velocity, so not all of them will fit.
- Roles rotate: Omar (PO), Saeed (SM), Yehia (Developer) for Sprint 2.
- DoD may tighten — at minimum we'll add “code is committed in small, regular pushes” after the retro feedback.

10. Conclusion

Sprint 1 of the UMS delivered the full MVP — 10 Must Have stories, 46 story points, all acceptance criteria met. The Scrum events (planning, daily standups, review, retro) gave us structure and the MoSCoW prioritisation kept us focused on what mattered. We had real challenges (US-02 overrun, CSS drift) but the retrospective gave us clear actions to take into Sprint 2.

Phase 2 in one sentence: the system works end-to-end, the process worked, and we know what to improve next time.